

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSAE309	Course Title:	Data Structures
EO Assessed:	CO2 - Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyse the efficiency of linear and binary search techniques.		
Student Name:	PRENA M	Reg. No.:	192524056
Section / Batch:	A	Date:	17/07/2026

Code of Conduct

I PRENA M [192524056]

(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: PRENA M

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear
Faculty In-charge		Course Coordinator		Program Director	
Signature: _____		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	

14 Two stacks in a single Array

Two stacks can be efficiently implemented using one array by allowing Stack 1 to grow from the left and stack 2 from the right.

Structure:

For an array of size N :

- Stack 1 grows from Index 0 toward $N-1$
- Stack 2 grows from Index $N-1$ toward 0
- overflow occurs only when $top_1 + 1 = top_2$

This approach ~~maximizes~~ memory utilization because unused space of one stack can be used by the other.

Fixed partitioning vs Dynamic Allocation

Feature	Fixed partitioning	Dynamic Allocation
Memory division	Array divided into fixed portions	Both stacks share entire array
Space utilization	Lower	Higher
overflow	can occur even when other partition has free space	occurs only when whole array is full
Flexibility	Low	High
complexity	simple	Slightly more complex
Suitable for	predictable workloads	varying workloads

Example:

If an array has 10 positions and Stack 1 needs 4 positions while Stack 2 needs only 2, fixed partitioning may waste one position. With dynamic sharing, stack 1 can use the available space.

Conclusion:

Dynamic allocation using two stacks growing toward each other is more memory-efficient and handles varying workloads better. Fixed partitioning is simpler but can cause internal unused space and premature overflow.

2) Simple queue vs priority - Based queue

A normal queue follows FIFO (First In, First Out). customers are served according to their arrival order.

A priority queue assigns each customer a priority. VIP customers are served before normal customers, even if they arrived later.

comparison.

Feature	Simple queue	priority Queue
Service order	Arrival order	priority first
Fairness	High	Lower for normal customers
VIP handling	not possible directly	easily supported.
waiting time for VIP	may be high	Reduced
waiting time for normal customers	predictable	May increase
complexity	Lower	Higher
Basic operation.	FIFO	priority-based

Analytical Example:

Suppose customers arrive as:

A $\rightarrow$ B $\rightarrow$ VIP $\rightarrow$ C

Simple Queue:

A $\rightarrow$ B $\rightarrow$ VIP $\rightarrow$ C

Everyone is served according to arrival time.

Priority Queue:

VIP $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C

The VIP customer gets faster service, but A & B have to wait longer.

Fairness

A simple queue provides better fairness because customers are served in their arrival order. A priority queue improves service for important customers but can cause starvation if high-priority customers continuously arrive.

Complexity:

For a simple queue, insertion and deletion can generally be performed in $O(1)$ using a suitable implementation.

For a priority queue:

- * unsorted Array $\rightarrow$ insertion $O(1)$, deletion $O(n)$
- * Sorted structure $\rightarrow$ insertion $O(n)$, deletion $O(1)$
- * Heap $\rightarrow$ insertion and deletion $O(\log n)$

A simple queue is best when fairness and equal treatment are important. A priority queue is better when certain customers need faster service, but it increases complexity and may increase the waiting time of normal customers.

Woj